

TECHX ENTERPRISE AI — HACKATHON MASTER GUIDE

Full Technical Architecture, Tech Stack, 5 Agent Tools & Judge Defense Cheat Sheet

1. The 30-Second Elevator Pitch

TechX Enterprise AI is an end-to-end conversational analytics platform that democratizes database access for non-technical users. Powered by LLM function calling, it turns natural language queries into safe, read-only SQL/NoSQL operations, renders dynamic 2D/3D visualizations and Mermaid ER diagrams in real time, provides 100% SQL transparency with an interactive Rerun Drawer, and logs audit data into an unstructured NoSQL document store — all wrapped in an interactive Three.js 3D database command center.

2. Complete Technology Stack Matrix

Component	Technology	Role & Key Highlights
Frontend UI	React 18 + Vite + Tailwind CSS	SPA UI, HMR, dark mode navy/slate glassmorphism palette.
3D & Animations	Three.js + Anime.js	3D database node backdrop reacting to scroll up/down & spring physics.
Data Visuals	Recharts + Mermaid.js	Bar/Line/Pie/Scatter/Area charts with manual switcher + ER & Flowchart SVG renderer.
Backend Server	Python 3.12 + FastAPI	High-performance async REST API, OpenAPI docs (/docs), Pydantic validation.
AI / LLM Model	Google Gemini 2.5 Flash	Function calling SDK + Local smart orchestrator regex fallback engine.
Relational DB	SQLite 3 + Supabase Cloud	7 domain tables (customers, products, orders, inventory) + Cloud PG adapter.
NoSQL & Audit Logs	Firebase + SQLite NoSQL	Cloud Firestore + Unstructured JSON document log store (login_logs & chat_logs).
Security & DevOps	Google OAuth 2.0 + Docker	Read-only AST validator, PyJWT token handler, Dockerfile, Pytest suite.

3. Deep-Dive Specification of the 5 Agent Tools

- **get_schema (schema_tool.py)**

Inspects relational database schemas (tables, columns, types, PKs, FKs) and NoSQL document collections. Returns structured JSON schema.

- **execute_query (query_tool.py)**

Validates & executes read-only SQL SELECT queries. Blocks non-read statements (INSERT, UPDATE, DELETE, DROP). Returns rows, count, execution time (ms).

- **generate_chart (chart_tool.py)**

Creates payloads for Bar, Line, Pie, Scatter, Area charts. Features manual chart-type switcher, accessible table fallback, and SVG download.

- **generate_flowchart (diagram_tool.py)**

Generates Mermaid.js definitions for Entity-Relationship Diagrams (erDiagram) and Process Flow Pipelines (graph TD). Renders zoomable SVG.

- **explain_data (explain_tool.py)**

Analyzes query outputs to produce natural language summaries, statistical highlights, and strategic recommendations.

4. End-to-End Execution Workflow

1. **Input Ingestion:** User inputs text query or speaks via browser Speech-to-Text audio Web API.
2. **Context Packaging:** Frontend packages current prompt + last 4 turns from app_state.db and posts to /api/chat.
3. **Schema Discovery:** LLM agent calls get_schema() to identify table columns, data types, and foreign keys.
4. **SQL Generation & Security Check:** Agent generates SQL SELECT statement; execute_query validator enforces read-only safety.
5. **Query Execution & Timing:** Query executes against SQLite/Supabase, returning JSON rows and query duration in ms.
6. **Visualization & Insights:** Agent calls generate_chart() for Recharts payload and explain_data() for analytical takeaways.
7. **NoSQL Document Logging:** Turn is logged into unstructured_logs.db under chat_logs collection.
8. **UI Card Render:** UI renders Chat Card + SQL Panel + Edit SQL Drawer + Interactive Chart + Insight Card.

5. Hackathon Judge Q&A; Defense — Winning Answers

Q1: How do you prevent SQL Injection or destructive operations (e.g. DROP TABLE)?

Answer: We enforce a strict Read-Only SQL AST Validator in query_tool.py. Before any SQL string reaches the database engine, it is stripped of comments, tokenized, and verified. If it contains any statement other than SELECT (e.g., INSERT, UPDATE, DELETE, DROP, ALTER), execution is rejected with a 400 Security Exception.

Q2: What happens if the LLM API key fails or network is offline?

Answer: We built an Intelligent Local Smart Orchestrator in agent.py. If the LLM API fails or no API key is present, our orchestrator dynamically inspects the schema using natural language regex rules to generate valid SQL SELECT queries, build Recharts payloads, construct Mermaid ER diagrams, and return structured responses with zero

downtime!

Q3: How do you handle unstructured audit data alongside relational databases?

Answer: We developed a dual database engine in `unstructured_logger.py`. While business data resides in relational tables (`ecommerce.db`), all login records (`login_logs`) and chat turns (`chat_logs`) are saved as unstructured JSON documents inside `unstructured_logs.db` using SQLite's native `json_extract()` functions.

Q4: How does your visualization system handle accessibility and user preference?

Answer: In `ChartViewer.jsx`, every chart component includes a Manual Chart-Type Switcher allowing users to switch between Bar, Line, Pie, Scatter, and Area views in real time. We also built an Accessible Data Table Fallback button that converts any chart into a high-contrast HTML table.

Q5: How did you test and verify your agent tools?

Answer: We implemented an automated Pytest test suite in `backend/tests/test_tools.py`. It tests all 5 tools (`get_schema`, `execute_query`, `generate_chart`, `generate_flowchart`, `explain_data`), verifying JSON schema structure, read-only SQL security enforcement, and Mermaid diagram output. All 6 tests pass in 0.24 seconds!